Predictive Modeling and Optimization in Logistics Systems Using Python

Case Study: SwiftRoute Logistics Pvt. Ltd.

Model Evaluation and Comparison

Three predictive model configurations were evaluated using the same test dataset: Linear Regression, a Decision Tree Regressor, and tunes Decision Tree Regressor. Performance was measured using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and the coefficient of determination (R^2).

Model	MAE (days)	RMSE (days)	R^2
Linear Regression	1.876	2.170	-0.069
Decision Tree	1.890	2.227	-0.126
Tuned Decision Tree	1.921	2.266	-0.166

Linear Regression achieved the lowest MAE and RMSE and the highest R^2 among the three configurations. However, all three models produced negative R^2 values, indicating that they performed worse on this test set than a baseline that predicts the mean delivery time from the training data.

Hyperparameter tuning was performed using five-fold-cross-validation on the training data. The best Decision Tree parameters were a maximum depth of 2 and a minimum of 10 samples per leaf. The tuned model did not improve test-set performance.

These results suggest that the selected input features do not adequetly predict the simulated delivery times. Because delivery times were randomly generated, the weak performance is an expected limitation of the dataset. The results

should not be interpreted as evidence about actual SwiftRoute Logistics operations.

Optimization Strategies and Recommendations

1. **Improve data collection:** Record actual delivery durations, planned delivery times, traffic conditions, route distances, fuel consumption, and reasons for delays.
2. **Review route efficiency:** Compare routes using distance, estimated travel time, and transportation costs. Test alternative route through simulation before implementation.
3. **Monitor delivery performance:** Track delivery success rates, delay rates, average delivery time, and cost per shipment using verified operational data.
4. **Improve resource allocation:** Use actual shipment volume, weight, vehicle capacity, and delivery locations to investigate more efficient vehicle assignments.
5. **Develop better predictive models:** Retrain and validate models using reliable historical records that contain meaningful predictors of delivery time.
6. **Validate improvements:** Compare baseline and proposed operations using measurable outcomes such as average delivery duration, delay rate, and transportation cost.

Python Implementation and Code Documentation

1. **Data Preparation**

   The dataset was loaded using Pandas. Delivery time was selected as the target variable, while distance, shipment weight, shipment volume, transportation cost, and delivery zone were selected as input features. The data was divided into training and testing sets using an 80:20 split.

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split

# Load the simulated logistics dataset
df = pd.read_csv("SwiftRoute_Logistics_Dataset.csv")
```

```
# Define input features and target variable
features = [
    "Distance_KM",
    "Shipment_Weight_KG",
    "Shipment_Volume",
    "Transportation_Cost",
    "Delivery_Zone"
]

X = df[features].copy()
y = df["Delivery_Time_Days"].copy()

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

print("Dataset shape:", df.shape)
print("Training records:", len(X_train))
print("Testing records:", len(X_test))
```

Explanation: The dataset contains 500 records and 9 columns. The split allocates 400 records for training and 100 for testing. The fixed random state makes the split reproducible when the same dataset and environment are used.

2. Model Training

A preprocessing pipeline was created to handle numerical and categorical features. One-hot encoding converted delivery zones into numerical indicators. Two regression models were then trained using the same training dataset.

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
```

```
numeric_features = [
    "Distance_KM",
    "Shipment_Weight_KG",
    "Shipment_Volume",
    "Transportation_Cost"
]

categorical_features = ["Delivery_Zone"]

preprocessor = ColumnTransformer(
    transformers=[
        ("numeric", "passthrough", numeric_features),
        ("zone", OneHotEncoder(handle_unknown="ignore"),
         categorical_features)
    ]
)

# Linear Regression model
linear_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("model", LinearRegression())
])

linear_model.fit(X_train, y_train)

# Decision Tree model
tree_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("model", DecisionTreeRegressor(
        max_depth=4,
        random_state=42
    ))
])

tree_model.fit(X_train, y_train)

print("Both models trained successfully.")
```

Explanation: Linear Regression provides a baseline model, while Decision Tree Regression can represent nonlinear relationships. The pipeline

ensures that categorical encoding is fitted on training data and then applied consistently to test data.

3. **Model Evaluation**

The trained models were evaluated on the same 100 test records. MAE and RMSE measure prediction error in days, while R^2 compares predictive performance with a baseline based on the mean target value.

R^2 compares predictive performance with a baseline based on the mean target value.

```
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

# Generate predictions
linear_pred = linear_model.predict(X_test)
tree_pred = tree_model.predict(X_test)

# Evaluate both models
def evaluate_model(name, actual, predicted):
    return {
        "Model": name,
        "MAE": mean_absolute_error(actual, predicted),
        "RMSE": np.sqrt(
            mean_squared_error(actual, predicted)
        ),
        "R2": r2_score(actual, predicted)
    }

results = pd.DataFrame([
    evaluate_model("Linear Regression", y_test, linear_pred),
    evaluate_model("Decision Tree", y_test, tree_pred)
])

display(results.round(3))
```

Actual results from this project:

Model	MAE (days)	RMSE (days)	R^2

Linear Regression	1.876	2.170	-0.069
Decision Tree	1.890	2.227	-0.126

Interpretation: Linear Regression performed slightly better than the initial Decision Tree. Both models had negative R^2 values, indicating that neither outperformed the training-mean baseline on the test set.

4. **Hyperparameter Tuning**

GridSearchCV was used to evaluate Decision Tree configurations through five-fold cross-validation on the training dataset. The tuning process tested different maximum tree depths and minimum numbers of samples per leaf.

```
from sklearn.model_selection import GridSearchCV

param_grid = {
   "model__max_depth": [2, 3, 4, 5, None],
   "model__min_samples_leaf": [1, 2, 5, 10]
}

grid_search = GridSearchCV(
   estimator=tree_model,
   param_grid=param_grid,
   scoring="neg_mean_absolute_error",
   cv=5,
   n_jobs=-1
)

grid_search.fit(X_train, y_train)

best_tree = grid_search.best_estimator_

# Evaluate the tuned model on the test set
tuned_pred = best_tree.predict(X_test)
```

```
tuned_mae = mean_absolute_error(y_test, tuned_pred)
tuned_rmse = np.sqrt(
   mean_squared_error(y_test, tuned_pred)
)
tuned_r2 = r2_score(y_test, tuned_pred)

print("Best parameters:", grid_search.best_params_)
print("Best cross-validation MAE:",
     round(-grid_search.best_score_, 3))
print("Test MAE:", round(tuned_mae, 3))
print("Test RMSE:", round(tuned_rmse, 3))
print("Test R2:", round(tuned_r2, 3))
```

Actual results from this project:

- Best parameters: maximum depth = 2; minimum samples per leaf = 10.
- Best cross-validation MAE: 1.985 days.
- Test MAE: 1.921 days.
- Test RMSE: 2.266 days.
- Test R^2: -0.166.

Interpretation: Hyperparameter tuning did not improve test performance. The tuned Decision Tree performed worse than Linear Regression on the held-out test set.

5. **<u>Reproducibility and Limitations</u>**

   The notebook contains the Python code used to train and evaluate the models. The dataset is simulated, and delivery times were randomly generated. Therefore, the model results demonstrate the predictive-modelling workflow but do not establish real-world forecasting accuracy for SwiftRoute Logistics.

```
# Verify dataset and target used in the analysis
print("Dataset records:", len(df))
print("Number of input features:", len(features))
print("Target variable:", y.name)
print("Missing values in selected inputs:", X.isnull().sum().sum())
print("Missing values in target:", y.isnull().sum())
```

Explanation: These checks helps document the dataset used in the analysis. The project uses simulated data, and delivery times were randomly generated. Consequently, the results demonstrate the modelling workflow rather than real-world forecasting.